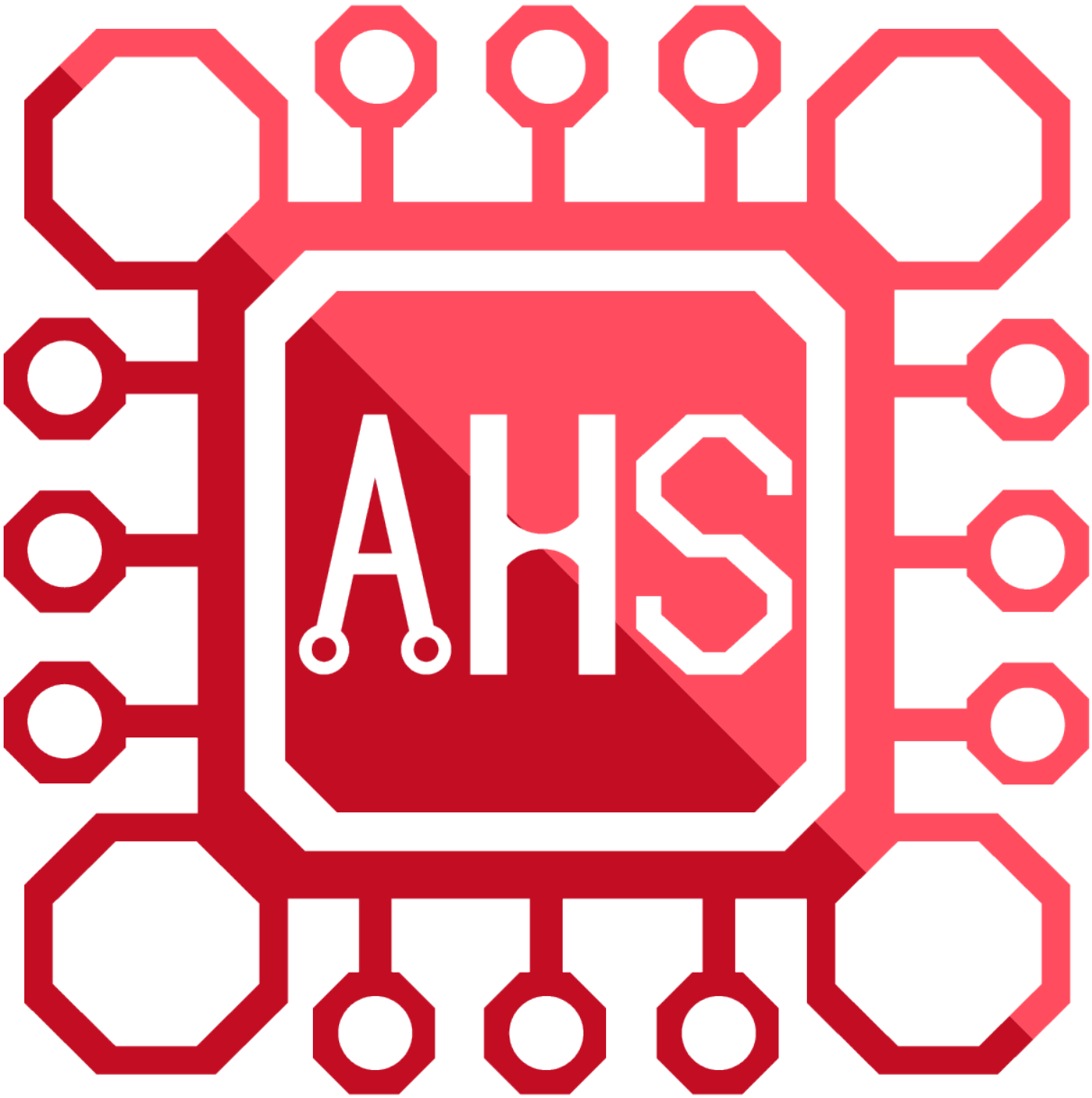


tutorial-asplos2025 Documentation

Versão 1.2024-preview

Massimo DiPierro

1 Overview **3**

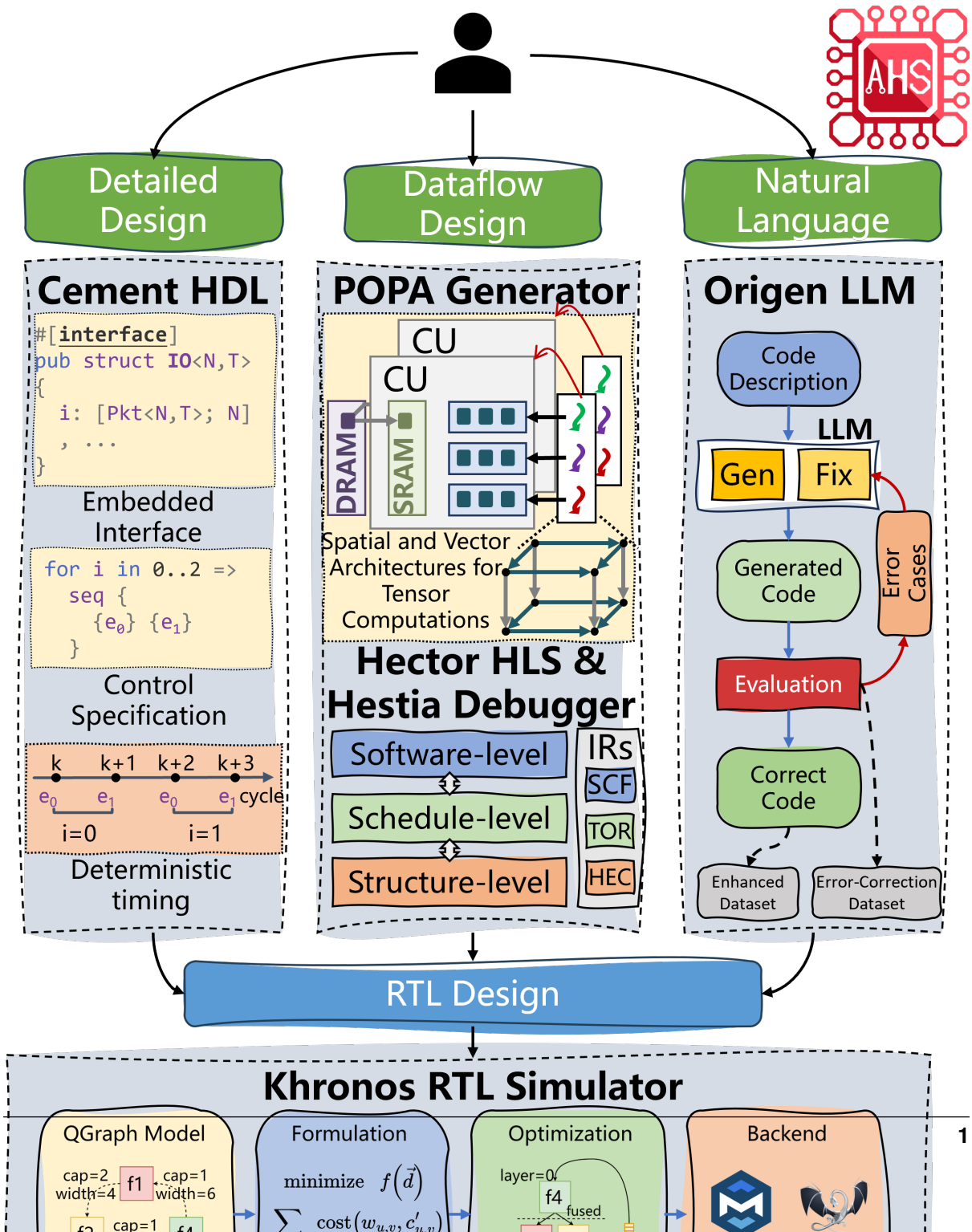
2 Schedule **5**

3 Organizers **7**

4 Related papers **9**

5 Git Repos **11**

5.1 Install Steps 11


AHS
Detailed Design
Dataflow Design
Natural Language
Cement HDL
#[interface]
pub struct IO<N,T>
{
i: [Pkt<N,T>; N]
, ...
}
Embedded Interface
for i in 0..2 =>
seq {
{e0} {e1}
}
Control Specification
k
k+1
k+2
k+3
e0
e1
e0
e1
cycle
i=0
i=1
Deterministic timing
POPA Generator
CU
CU
DRAM
SRAM
Spatial and Vector Architectures for Tensor Computations
Hector HLS & Hestia Debugger
Software-level
Schedule-level
Structure-level
IRs
SCF
TOR
HEC
Origen LLM
Code Description
LLM
Gen
Fix
Generated Code
Error Cases
Evaluation
Correct Code
Enhanced Dataset
Error-Correction Dataset
RTL Design
Khronos RTL Simulator
QGraph Model
cap=2
width=4
f1
cap=1
width=6
cap=1
Formulation
minimize $f(\vec{d})$
$\sum \text{cost}(w_{u,v}, c'_{u,v})$
Optimization
layer=0
f4
fused
Backend

Chapter 1

Overview

Compared to software design, hardware design is more expensive and time-consuming. This is partly because software community has developed a rich set of modern tools to help software programmers to get projects started and iterated easily and quickly. However, the tools are seriously antiquated and lacking for hardware design. Modern digital chips are still designed manually using hardware description language such as Verilog or VHDL, which requires low-level and tedious programming, debugging, and tuning. In this tutorial, we will introduce Agile Hardware Specialization (AHS) : A toolbox for Agile Chip Front-end Design.

The tutorial will highlight the methodology and open source tools in AHS for both chip design and verification. From the design perspective, AHS present three ways that use different programming interfaces and target different scenarios, including; 1) a multi-level hardware intermediate representation based high-level synthesis flow, which uses C and C++ as the programming language; 2) an embedded hardware description language, which uses Rust as the programming language; 3) a large language model (LLM)-powered hardware design flow, which uses natural language as the programming language. These three different methodologies exhibit different trade-offs in productivity and PPA (performance, power, and area) for chip design. From the verification perspective, we will present agile simulation and debugging tools, which can check the functional and performance behaviors of the hardware. The attendees will learn the methodology, design automation fundamentals, and software tools of AHS.

Chapter 2

Schedule

Chapter 2

Chapter 3

Organizers

Chapter 3

Chapter 4

Related papers

Chapter 4

Chapter 5

Git Repos

5.1 Install Steps